Appendix: Artifact Description

Artifact Description (AD)

Paper ID: pap142

I. Part 1: Overview of Contributions and Artifacts

A. Paper's Main Contributions

- C1 - Target-centric execution. TempGNN introduces Target Dependency Packets (TDPs) to express only the temporally valid states needed by a target update, exposing branch-level parallelism while preserving ordered target state commitment.
- C2 - Dependency-driven construction. TempGNN combines temporal sampling, dependency expansion, and data loading in an on-the-fly DDTC pipeline to reduce target-irrelevant work and off-chip traffic.
- C3 - Overlap-aware execution. TempGNN detects shared dependency packets among concurrent targets and uses OATS/PHLE reuse while retaining order-preserving target updates.
- C4 - U280 evaluation. The paper evaluates the execution behavior, performance, energy, ablations, and sensitivity of TempGNN across four TGNN models and six temporal datasets, including comparisons with software and FPGA baselines.

B. Computational Artifacts

Primary code repository:
https://github.com/TempGNN-Program/TempGNN

Reviewer-downloadable package:
https://github.com/TempGNN-Program/TempGNN/raw/main/ae_export/pap142_tempgnn_sc26_ae_u280.tgz

The version-specific DOI will be inserted before the SC26 artifact freeze on

August 25, 2026. GitHub alone is not the persistent archive; the frozen release

will also be deposited in Zenodo, Figshare, or the conference artifact

repository. The repository and package are licensed under Apache-2.0.

- A1: TempGNN reference model, TDP/DDTC/OATS tests, and real edge-stream profiling.
- A2: TempGNN plus independent paper-based MATG, ViTeGNN, and RTGA U280 forward paths, common XRT host, xclbins, reports, and measurement workflow.
- A3: Source-labeled paper-reference data and deterministic CSV/SVG generator.

Artifact	Contributions supported	Related paper elements
A1	C1, C2, C3	TDP correctness and mechanism behavior; Fig.4(a), Fig.13, Fig.14 context
A2	C2, C3, bounded evidence for C4	U280 functional evidence and fresh Fig.11/Fig.12-shaped diagnostic tables
A3	Audit record for C4	Fig.2, Fig.4(a), Fig.9(b), and Fig.10-Fig.14

C. Measurement Boundary

The packaged TempGNN board logs, timing report, and layout are historical U280

sanity evidence. Artifact A2 additionally executes four distinct xclbins and

collects fresh U280 latency, total-board power, clock, checksum, and input

provenance rows. MATG, ViTeGNN, and RTGA are independently written,

paper-based forward-path reproductions, not the baseline authors' complete

source trees.

The fresh four-system path is deliberately marked as bounded and diagnostic.

It uses 8-dimensional Q10 kernels, deterministic stand-in weights, and

8,192-event real-data prefixes. The paper uses complete model configurations,

default 32-bit floating point, trained checkpoints, full evaluation streams,

and post-route Vivado power estimates. Artifact A3 regenerates packaged

paper-reference figures from documented numeric inputs; it does not execute

the external baseline stacks. The repository therefore sets `results_reproduced_eligible: false` and does not currently assert the Results

Reproduced badge.

II. Part 2: Artifact Identification

III. Computational Artifact A1

A. Relation to Contributions

A1 is the inspectable software specification for TDP construction, temporal

dependency traversal, DDTC scheduling, PHLE packet reuse, and OATS overlap

handling. It supports C1–C3 independently of FPGA availability and exposes

the counters used to reason about branch parallelism, packet overlap, reuse,

collisions, synchronization stalls, and off-chip traffic.

B. Expected Results

The unit suite must pass all 28 tests. The TDP implementation must agree with a

chronological reference update, fixture generation must be deterministic, and

artifact paths/provenance must pass their consistency checks. Optional Q14

profiling produces nonnegative, well-formed per-dataset/model statistics from

real temporal edge streams. These checks substantiate the functional behavior

of the execution mechanisms in C1–C3; they are not performance measurements

of the full paper system.

C. Expected Reproduction Time (in Minutes)

Step	Time	Notes
Setup	5-10	Create a Python environment and install requirements.txt
Execution	less than 2	Unit tests; optional full Q14 download/profile takes 10-40
Analysis	less than 2	Inspect test output and generated CSV/Markdown summaries

D. Artifact Setup

1) Hardware: Any Linux x86_64 machine is sufficient. No FPGA is required for A1.

2) Software:

- Python 3.10 or newer: https://www.python.org/.
- GNU Make and Bash: https://www.gnu.org/software/make/.
- NumPy 1.23 or newer for the complete software workflow.
- The exact recorded environment is in ENVIRONMENT.md.

3) Datasets and Inputs: Unit tests generate deterministic fixtures. Optional real-edge profiling uses

TGL edge streams from https://github.com/amazon-science/tgl. The package also

contains fixed 8,192-event prefixes for WK, MC, RT, LM, WT, and GT under

external/u280_dataset_samples/, each with its source URL, selection rule, and

SHA256 metadata.

4) Installation and Deployment:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

E. Artifact Evaluation

The default dependency chain is:

```
T1: create environment
  -> T2: run 26 unit tests
  -> T3: optionally fetch real edge streams
  -> T4: profile TDP/DDTC/OATS behavior
  -> T5: generate CSV and Markdown summaries
```

Commands:

```
python3 -m unittest discover -s tests
make data
make q14
```

Q14 defaults to WIKI, MOOC, and REDDIT; models are JODIE, TGAT, TGN, and APAN;

the target batch size, fanout, and depth are recorded in the output CSVs.

F. Artifact Analysis

The test command must finish with Ran 28 tests and OK. Optional Q14 outputs

are written to:

```
results/q14_real_tgl_edges/q14_dataset_model_summary.csv
results/q14_real_tgl_edges/q14_batches.csv
results/q14_real_tgl_edges/q14_summary.md
```

The summary reports packet hit/reuse rates, collisions, synchronization stalls,

memory traffic, and a 225 MHz cycle-model latency. It must not be interpreted

as a fresh board timing result.

IV. Computational Artifact A2

A. Relation to Contributions

A2 makes the TempGNN forward path and the paper-based MATG, ViTeGNN, and RTGA

mechanisms executable on the same Alveo U280. It provides functional U280

evidence for C2–C3 and a controlled diagnostic comparison related to C4.

Each implementation has a separate source top, synthesis result, xclbin, and

provenance record. Missing or byte-identical xclbins cause preflight failure.

B. Expected Results

Preflight must report four distinct xclbin SHA256 values. Every hardware row

must pass the software golden output and repeat-consistency checks, contain a

real-input URL and hash, and record the xclbin link request, Vivado-connected

kernel clock, and post-route WNS/TNS.

With 3 repetitions, the full matrix contains 288 raw rows:

```
4 implementations x 6 datasets x 4 models x 3 repetitions
```

The workflow derives fresh Fig.11/Fig.12-shaped CSV/SVG tables and a numerical

tolerance report. A tolerance failure is retained as evidence and is not

replaced with packaged reference values. Because the configuration is not

paper-equivalent, even a numerical pass is diagnostic only.

C. Expected Reproduction Time (in Minutes)

Step	Time	Notes
Setup	10-20	Receive reviewer account, source XRT, verify U280
Execution	30-90	Direct run of four packaged xclbins, 3 repetitions
Analysis	less than 5	Automatic aggregation, figures, tolerance and provenance
Optional rebuild	240-480 per xclbin	Not required for review and outside the direct-run path

D. Artifact Setup

1) Hardware:

- AMD/Xilinx Alveo U280, device `xcu280-fsvh2892-2L-e`.
- Platform `xilinx_u280_gen3x16_xdma_1_202211_1`.
- The authors provide a time-bounded account on a matching U280 through the private SC submission channel. No credentials are stored in the artifact.

2) Software:

- Ubuntu 22.04 x86_64, Linux 5.15 in the recorded run.
- XRT 2.16.204: `https://github.com/Xilinx/XRT`.
- Vitis/Vivado 2023.2 is required only to rebuild: `https://www.xilinx.com/support/download.html`.
- GCC 11.4, GNU Make 4.3, Bash, Python 3.10 or newer.

3) Datasets and Inputs: The direct workflow uses bundled, timestamp-sorted 8,192-event prefixes of

Wikipedia, MOOC, Reddit, LastFM, WikiTalk, and GDELT. Each prefix has source and

hash metadata. Fixtures are generated for JODIE, TGN, TGAT, and APAN with

batch size 1,000. Synthetic inputs are permitted only for C-sim and are

rejected by the core workflow. The repository and frozen archive retain the

exact generated fixture metadata and goldens under `results/generated_u280_comparison_fixtures/` for direct hash inspection.

4) Installation and Deployment:

```
source /opt/xilinx/xrt/setup.sh
source
↪  /tools/Xilinx/Vitis/2023.2/settings64.sh
↪  # rebuild only
make u280-core-preflight
```

The reviewer-facing files are under `artifacts/u280/`; source and mechanism

mapping are under `hardware/` and `hardware/baselines/`.

E. Artifact Evaluation

The direct-run dependency chain is:

```
T1: preflight hashes, files, source revisions,
↪  and four xclbins
  -> T2: generate deterministic real-prefix
  ↪  fixtures and software goldens
  -> T3: run each xclbin with a common XRT
  ↪  host
  -> T4: sample gated U280 power and record
  ↪  per-repetition raw rows
  -> T5: validate completeness, checksums,
  ↪  energy, and timing-closed clocks
  -> T6: aggregate rows and derive diagnostic
  ↪  Fig.11/Fig.12 tables
  -> T7: compare with source-labeled
  ↪  paper-reference values
```

Commands:

```
make u280-core-preflight
make ae-core-u280 U280_CORE_DEVICE=0
↪  U280_CORE_REPETITIONS=3
```

The checked parameters are in `configs/u280_core_reproduction.json`: requested

225 MHz, achieved-clock tolerance 0.5 MHz, batch size 1,000, six datasets, four

models, three repetitions, 8-dimensional Q10 tensors, and bounded real-data

prefixes.

F. Artifact Analysis

Fresh results are written to a new timestamped directory:

```
results/reviewer_u280_runs/<run-id>/provenanc⌋
↪  e.json
results/reviewer_u280_runs/<run-id>/raw/*/mea⌋
↪  surements.csv
results/reviewer_u280_runs/<run-id>/baselines⌋
↪  _u280/
results/reviewer_u280_runs/<run-id>/derived_c⌋
↪  omparison_figures/
results/reviewer_u280_runs/<run-id>/verificat⌋
↪  ion.json
results/reviewer_u280_runs/<run-id>/verificat⌋
↪  ion.md
```

The package includes smoke run `pap142_u280_smoke_20260715T052052Z` and final

run `pap142_u280_measured_20260715T052052Z`; the latter contains 288 measured

rows and is the run cited by the generated AE report.

Latency is XRT launch-to-completion kernel time. Power is total U280 board power

sampled only during the gated repeated-kernel window. Energy in millijoules is

`latency_ms * power_w`. The orchestrator rejects incomplete matrices, failed

goldens, inconsistent energy, synthetic core inputs, and incomparable achieved

clocks. Post-route timing, utilization, HLS, route, build, and xclbin metadata

evidence is staged with SHA256 records.

V. Computational Artifact A3

A. Relation to Contributions

`A3` preserves the numerical inputs used to plot the paper evaluation and makes

the transformation to CSV/SVG inspectable and deterministic. It is an audit

artifact for `C4`, not a substitute for executing `A2` or external CPU/GPU

baseline stacks.

B. Expected Results

The command regenerates nine paper-reference figures and matching CSV files.

Every input row is labeled as either an exact author-workbook cell or an

axis-calibrated recovery from an author vector export. No value is backsolved

from a reported mean or silently replaced by a fresh U280 row.

Expected summary values include:

```
TempGNN vs TGLite-CPU: 132.80x
TempGNN-G vs TGLite-CPU: 10.85x
Cascade vs TGLite-CPU: 5.22x
TempGNN vs MATG, explicit plotted AVG: 7.7889x
Paper prose for TempGNN vs MATG: 7.6x
Energy, Cascade/TempGNN, explicit plotted AVG:
↪  33.545x
w/o DDTC normalized time: 3.08x
w/o OATS normalized time: 1.77x
```

C. Expected Reproduction Time (in Minutes)

Step	Time	Notes
Setup	less than 5	Same Python environment as `A1`
Execution	less than 1	Deterministic CSV/SVG generation
Analysis	2-5	Inspect manifest, source labels, and expected summaries

D. Artifact Setup

1) Hardware: Any Linux x86_64 machine; no FPGA or GPU is required.

2) Software: Python 3.10 or newer. Core CSV/SVG generation uses the Python standard library.

3) Datasets and Inputs: The sole numeric input is `reference_inputs/paper_figure_values.csv` with 668

source-labeled rows. `reference_inputs/README.md` records source hashes,

workbook cell or vector geometry locators, uncertainty, and the Fig.11

plot/prose discrepancy.

4) Installation and Deployment: Use the environment prepared for `A1`; no additional deployment is required.

E. Artifact Evaluation

```
T1: read and validate the source-labeled
↪  reference table
  -> T2: group rows by figure, dataset, model,
  ↪  and solution
  -> T3: emit matching CSV and SVG files
  -> T4: write a manifest and combined source
  ↪  table
  -> T5: run consistency tests
```

Command:

```
python3 -m scripts.reproduce_paper_figures
```

F. Artifact Analysis

Outputs are under `results/paper_reproduction/`, including Fig.2, Fig.4(a),

Fig.9(b), and Fig.10-Fig.14 CSV/SVG pairs, `all_figure_data.csv`, and

`figure_data_manifest.csv`. The manifest identifies the input file and data

status for every figure. The explicit Fig.11 AVG bar and the rounded 7.6x prose

statement are both preserved rather than forced to agree.